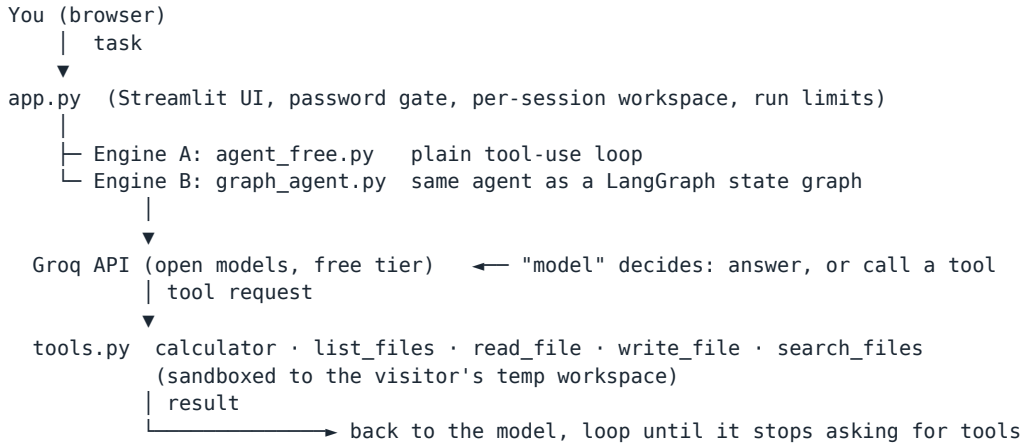
End-to-End Guide: Autonomous AI Agent (Groq + LangGraph + Streamlit)

From zero to a live, tested, auto-restarting app. Every step below has been used on the real project.

0. What you're building

An assistant that **plans, calls tools, and finishes multi-step tasks**, with a live UI.



The six build steps → where they live

Step	What	Where
1	Pick the task and tools	`tools.py`
2	Describe each tool as JSON schema	`TOOLS` list in `tools.py`
3	Call the model with tools attached	`client.chat.completions.create(..., tools=...)` in `agent_free.py`
4	Run the requested tool, return the result	`tool` messages in `agent_free.py`
5	Loop until no more tool requests	`if not msg.tool_calls: return`
6	Logging, retries, guardrails	`logging`, `run_with_retry`, step/token caps, sandbox

1. Prerequisites

- Python 3.12 (3.11 also works): `python --version`
- Git: `git --version`
- A GitHub account and a Streamlit Community Cloud account (sign in with GitHub)
- A free Groq API key: https://console.groq.com → **API Keys → Create API Key** (shown once; copy it)

2. Get the code

Unzip `autonomous-agent.zip`, then **work inside the folder** so `app.py` is at the root:

```
cd autonomous-agent
```

Files (hidden folders `.github/` and `.streamlit/` are part of it, so show hidden files if you can't see them):

```
app.py                  Streamlit UI
agent_free.py           plain agent loop (Groq/Ollama)
graph_agent.py          LangGraph version
tools.py                tool schemas + sandboxed implementations
agent.py                optional: same loop on the Claude API
scripts/keepalive.py    headless-browser wake-up check
tests/                  offline tests (no key needed)
requirements.txt        what the deployed app installs
requirements-dev.txt    + pytest
.streamlit/             config + secrets template
.github/workflows/      ci.yml and restart-app.yml
```

---

3. Run it locally

```
python -m venv .venv
.venv\Scripts\activate            # Mac/Linux: source .venv/bin/activate
pip install -r requirements-dev.txt
```

Create your local secrets file (it is git-ignored):

```
copy .streamlit\secrets.toml.example .streamlit\secrets.toml     # Mac/Linux: cp
```

Edit `.streamlit/secrets.toml`:

```
GROQ_API_KEY = "gsk_your_real_key"
# APP_PASSWORD = "optional-password"
```

Run the tests (no network, no key needed):

```
pytest -q          # expect: all passed
```

Start the app:

```
streamlit run app.py        # opens http://localhost:8501
```

Quick CLI check without the UI:

```
# Windows cmd:   set GROQ_API_KEY=gsk_...
# PowerShell:    $env:GROQ_API_KEY="gsk_..."
# Mac/Linux:     export GROQ_API_KEY=gsk_...
python agent_free.py "Write hello.txt containing 12*34, then read it back"
```

What you should see: a trace of `write_file` → `read_file` → a final answer; files appear in the sidebar.

---

4. Push to GitHub

1. github.com → **New repository** (e.g. `Autonomous-agent`). Leave "Add a README" **unticked**.

2. From inside the project folder:

```
git init
git add .                       # NOT "git add README.md": that stages one file only
git status                      # confirm app.py, tools.py, .github/, .streamlit/, tests/ are
  listed
git commit -m "Autonomous agent"
git branch -M main
git remote add origin https://github.com/<you>/<repo>.git
git push -u origin main
```

3. Refresh GitHub: you should see `app.py`, `requirements.txt`, and `.github/workflows/` with two YAML files.

> *Never commit your key.* `.gitignore` *already excludes* `.streamlit/secrets.toml`*.*

Rule for later updates: commit all changed files **together** in one commit (`git add .`). Pushing files one by one can deploy a half-updated app (see Troubleshooting).

---

5. Deploy on Streamlit Community Cloud

1. Go to https://share.streamlit.io and sign in with GitHub (authorize access to your repo if asked).
2. **Create app** → deploy from an existing repo:
   - Repository: `<you>/<repo>`
   - Branch: `main`
   - Main file path: `app.py`
   - App URL: choose a subdomain
3. **Advanced settings**:
   - Python version: 3.12
   - Secrets (TOML):

```
GROQ_API_KEY = "gsk_your_real_key"
APP_PASSWORD = "pick-a-password"
```

4. **Deploy.** The first build takes a few minutes (it installs `requirements.txt`).
5. Open your URL, enter the password, click an example task, and watch the trace stream in.
6. Test **both engines** from the sidebar (Plain loop and LangGraph).

Updating the live app: `git push` to `main`. Streamlit redeploys automatically (usually under two minutes). Use **Manage app → Reboot app** if it looks stuck. Edit secrets under **Manage app → Settings → Secrets**.

(Streamlit may rename menu items over time; the flow stays the same.)

---

6. Auto-restart and keep-awake every 6 hours

`.github/workflows/restart-app.yml` runs every 6 hours (UTC) and on demand. It (1) pushes an empty commit so Streamlit redeploys, then (2) runs `scripts/keepalive.py` in a headless browser, which clicks the wake-up button if the app was hibernating and checks the app renders.

One-time setup in your GitHub repo:

1. **Settings → Secrets and variables → Actions → Variables → New repository variable**
   - Name: `APP_URL` · Value: your app URL, e.g. `https://your-app.streamlit.app`

2. **Settings → Actions → General → Workflow permissions → Read and write permissions** → Save.
3. **Actions** tab → **Restart and keep app awake** → **Run workflow**. Open the run and confirm it prints either "clicked the wake-up button" or "app was already awake", then "App is up and rendering."

Options and caveats:

- Keep-awake only, no restart: set `RESTART_APP: "false"` at the top of the workflow.
- Each restart wipes visitors' temporary files and adds an empty commit (about 4 per day).
- If the empty-commit restart doesn't redeploy on your account, use **Manage app → Reboot app**, or switch to keep-awake only.
- If Streamlit changes its page layout, update the button text/selectors in `scripts/keepalive.py`.

---

7. How the pieces work (read this to explain it in an interview)

The loop (`agent_free.py → run_agent`):

1. Send messages + tool schemas to the model.
2. If the reply has no tool calls → that's the final answer; stop.
3. Otherwise run each requested tool, append the results as `tool` messages, go to 1.
4. Stop anyway at the step cap, token budget, or on a rate limit.

Guardrails already built in

- Step cap (default 10-12) and token budget (60k)
- Tool allowlist; unknown tools return an error to the model instead of crashing
- Per-visitor temp workspace; `../` path traversal blocked; file size and count limits
- Tool output truncated to 4,000 characters
- Tool crashes retried; recoverable errors returned to the model so it can try another approach
- Malformed JSON arguments and malformed tool calls are corrected, not fatal
- Rate-limit and model-not-found errors become friendly messages
- Per-session run cap (15), cooldown, optional `APP_PASSWORD`

Why two engines: the plain loop shows exactly what an agent is; LangGraph expresses the same thing as `agent → tools → agent → END`, which is the foundation for adding planners, checkpoints, and human approval.

Models: the default is `openai/gpt-oss-120b`. The sidebar lists models fetched live from Groq for your key, so a retired model doesn't break the app. For higher free-tier limits, pick a smaller one such as `openai/gpt-oss-20b`.

---

8. Extend it: add a new tool

Example: a `current_time` tool.

1. `tools.py`**: implementation** (workspace is always the first argument):

```
def current_time(ws: Path) -> str:
    from datetime import datetime, timezone
    return datetime.now(timezone.utc).isoformat()

IMPLS["current_time"] = current_time
```

2. `tools.py`**: schema** (append to `TOOLS`):

```
{
    "name": "current_time",
    "description": "Get the current UTC date and time.",
    "input_schema": {"type": "object", "properties": {}},
},
```

The plain loop picks it up automatically.

3. `graph_agent.py`**:** add a wrapper inside `build_graph` and include it in `tools`:

```
@tool
def current_time() -> str:
    """Get the current UTC date and time."""
    return _safe("current_time")

tools = [calculator, list_files, read_file, write_file, search_files, current_time]
```

4. Add a test in `tests/test_tools.py`, run `pytest -q`, then `git add . && git commit && git push`.

Good next tools

- **Web search** (e.g. a search API): the biggest capability jump. Keep results short and truncate output.
- **Human approval before** `write_file`**:** show a confirm button in `app.py` before executing writes.
- **LangGraph checkpointing** (`MemorySaver`) for resumable runs, and a **planner node** that stores the plan in graph state.

---

9. Troubleshooting (all of these happened on this project)

Symptom	Cause	Fix
Only `README.md` on GitHub	Ran `git add README.md` (the quick-setup snippet)	`git add .` → `git commit` → `git push`
`NotFoundError ... model_not_found`	Groq renamed/retired the model	Pick another model in the sidebar. Or update `PREFERRED_MODELS` and the default in `agent_free.py`, push
`ImportError: cannot import name 'PREFERRED_MODELS'`	`app.py` was deployed while `agent_free.py` was still the old version (files pushed separately)	Make sure both are committed, push, then **Reboot app**. It clears itself once all files match
"GROQ_API_KEY is missing"	Secret not saved	Manage app → Settings → Secrets → add it, save
"Rate limit hit" message	Free-tier limits	Wait a minute, or choose a smaller model in the sidebar
App shows "get this app back up"	Hibernated after inactivity	Click the button, or let the 6-hour workflow wake it
Files the agent wrote disappeared	Workspaces are temporary (restart/sleep wipes them)	Use the sidebar **Download** buttons
Workflow fails: `APP_URL is not set`	Variable missing	Add it under Settings → Secrets and variables → Actions → **Variables** (not Secrets)

Symptom	Cause	Fix
Workflow fails on `git push`	Workflow permissions read-only	Settings → Actions → General → Read and write
CI fails on GitHub	Missing dependency or a test failure	Open the failed run's log; run `pytest -q` locally to reproduce
Build error on Streamlit	Package problem in `requirements.txt`	Manage app → logs; fix the line, push

---

10. Pre-launch checklist

- ☐ `pytest -q` passes locally
- ☐ Both engines work on the live app
- ☐ `APP_PASSWORD` set (or deliberately removed for a public demo)
- ☐ `APP_URL` variable and workflow permissions configured; one manual workflow run succeeded
- ☐ No keys in the repo (`git log -p | grep gsk_` returns nothing)
- ☐ You know the free-tier limits: expect occasional rate-limit messages if traffic spikes
- ☐ You can explain: the loop, the guardrails, why a sandbox per visitor, and why two engines